v1.0

LOGLAR NE KADAR GUVENILIR?

Kernel-User Telemetri Desenkronizasyonu

Detection Engineering Perspektifi

FOCUS AREA	DETECTION TYPE	PLATFORM
Process Telemetry	**Kernel-User Desync**	**Windows ETW**

Birkan KES

Subat 2026

RESEARCH ACTIVE

Yönetici Özeti

Modern güvenlik izleme sistemleri, süreçlere dair kararlarını büyük ölçüde log verilerine dayandırır. Ancak bu veriler her zaman aynı zaman noktasını ve aynı gerçekliği temsil etmez. Windows ortamlarında, süreç oluşturma anında çekirdek tarafından kaydedilen bilgiler ile süreç çalışırken kullanıcı-modu araçları tarafından okunan veriler arasında anlamlı farklılıklar oluşabilir.

Bu fark, güvenlik araçlarının süreçleri yanlış veya eksik yorumlamasına neden olabilecek bir kör nokta yaratır. Özellikle süreç ağaçları ve komut satırı bilgilerine aşırı güvenen tespit yaklaşımları, saldırganlar tarafından manipüle edilebilen kullanıcı-modu verilerine dayanarak yanıltılabilir.

Sonuç: Güvenlik logları çelişkili ama kendi içinde tutarlı iki farklı gerçeklik gösterebilir. Çekirdeğin gördüğü ile güvenlik araçlarının gösterdiği aynı olmak zorunda değildir.

Bu araştırma, süreç oluşturma telemetrisindeki bu uyumsuzluğun pratikte nasıl istismar edilebildiğini ve tek kaynağa dayanan tespit stratejilerinin neden yetersiz kaldığını ortaya koymaktadır. Bulgular, kernel-temelli verilerin korunması ve farklı telemetri kaynaklarının birlikte değerlendirilmesinin, tespit güvenilirliğini artırmak için kritik olduğunu göstermektedir.

Giriş

Siber güvenlikte çoğumuzun kabul ettiği bir alışkanlık var: loglara bakarız ve sistemde olan biteni oradan anlamaya çalışırız.

- **Bir süreç zararsız mı yoksa şüpheli mi?**
- **Bu binary gerçekten hangi komut satırıyla başlatıldı?**
- **Bir alarm neden tetiklendi ya da neden tetiklenmedi?**

Bu soruların cevaplarını çoğu zaman Sysmon, EDR ve SIEM üzerinden toplanan telemetriye bakarak veririz. Bu yaklaşımın arkasında bir varsayım yatar: loglar, sistemde gerçekte olanı doğru ve eksiksiz biçimde yansıtıyordur. Ancak pratikte bu varsayım her zaman doğru değildir. Özellikle Windows mimarisinde, aynı olaya ait farklı telemetri kaynakları farklı zamanlarda ve farklı katmanlarda üretildiği için, loglar her zaman tek ve tutarlı bir hikâye anlatmaz.

ETW (Event Tracing for Windows) üzerine yapılan önceki analizler, çekirdek (kernel) ile kullanıcı-modu (user-mode) arasında üretilen telemetrinin zaman zaman birbirinden koptuğunu açıkça göstermektedir. Parent process spoofing gibi tekniklerde, çekirdeğin süreç hakkında bildiği bilgiler ile kullanıcı-modu araçlarının sunduğu görünüm arasında anlamlı farklar oluşabilmektedir. Bu fark, olayın kendisinden çok, olayın nasıl gözlemlendiğiyle ilgilidir.

Bu tutarsızlık sadece süreç ağacını mı etkiliyor, yoksa sürecin kendisine dair gördüğümüz her şey sorgulanmalı mı?

Bu noktada soru doğal olarak şuraya evrilir:
Hangi loglar gerçekten güvenilir?
Hangileri yalnızca anlık bir görünümü temsil eder ve manipülasyona açıktır?

Process Environment Block (PEB) manipülasyonu, bu sorunun etrafında dönen daha geniş bir problemin yalnızca bir örneğidir. Asıl mesele, bir sürecin oluşturulduğu anda çekirdek tarafından kayda geçen bilgiler ile süreç çalışırken güvenlik araçlarının referans aldığı kullanıcı-modu verileri arasındaki farktır. Süreç oluşturulurken çekirdek tarafından kaydedilen bazı alanlar olaydan sonra değiştirilemezken, süreç çalışırken kullanıcı-modu bellekten okunan bazı bilgiler yeterli erişimle sonradan değiştirilebilir.

Bu yazı, tek bir tekniği anlatmayı değil; loglara ne kadar ve hangi bağlamda güvenmemiz gerektiğini sorgulamayı amaçlamaktadır. SOC analistleri, detection engineer'lar, red team'ler ve Windows iç yapısına ilgi duyan herkes için ortak bir soruya odaklanır:

Gerçek hangi katmanda duruyor ve biz o gerçeği hangi loglara bakarak yorumluyoruz?

Kernel ve User-Mode Telemetri Arasındaki Ayrım

Aynı sürece ait telemetri, kaynağına ve gözlem zamanına bağlı olarak farklı katmanlardan gelir. Buradaki fark doğruluktan çok bağlam farkıdır: bazı loglar sürecin oluşturma anını, bazıları ise çalışma zamanındaki durumu yansıtır. Bu ayrım gözetilmediğinde, telemetri kolayca yanlış yorumlanabilir

Kritik soru: Eğer bazı loglar sürecin oluşturma anını, bazıları ise süreç çalışırkenki durumu gösteriyorsa, tespit ve analiz süreçlerinde hangisini referans alıyoruz?

Hızlı Karşılaştırma

Telemetri Kaynağı	Gözlem Katmanı	Zaman Bağlamı	Güvenilirlik Perspektifi
Sysmon Event ID 1	Kernel destekli (hibrit)	Oluşturma anı	Orta–Yüksek
User-mode Araçlar	User-mode	Çalışma zamanı	Düşük–Orta
EDR Çözümleri	Hibrit (ürüne bağlı)	Creation+Runtime	Değişken

Detaylı Analiz

☐ Sysmon Event ID 1

- **Katman:** Kernel destekli (hibrit)
- **Zaman:** Süreç oluşturma anı
- **Tetikleme:** Kernel callback tetiklenmesiyle başlar; bazı alanlar user-mode kaynaklardan zenginleştirilir.
- **Güçlü yön:** Süreç başlangıç anını erken yakalar, oluşturma sonrası değişikliklerden etkilenmez.
- **Sınırlama:** Oluşturma anından sonra yapılan metadata değişiklikleri görünmez; kısa zamanlama pencerelerinde kernel–user tutarsızlığı oluşabilir.

☐ User-mode Araçlar (Process Explorer, Task Manager)

- **Katman:** User-mode
- **Zaman:** Çalışma zamanı
- **Tetikleme:** Süreç çalışırken PEB üzerinden metadata okur
- **Güçlü Yön:** Mevcut durumu gösterir, anlık durum kontrolü
- **Sınırlama:** Okunan veriler değiştirilebilir bellek yapılarından gelir, manipülasyona açıktır

□ EDR Çözümleri

- **Katman:** Hibrit (Kernel + User)
- **Zaman:** Creation + Runtime (sürekli izleme)
- **Tetikleme:** Kernel callback'leri, ETW provider'ları ve ek sensörler ile süreç ilişkileri izlenir.
- **Güçlü Yön**: User-mode zenginleştirmeye göre daha erken ve sağlam gözlem noktası
- **Sınırlama:** UI katmanı user-mode veriye dayanabilir, korelasyon derinliği ve ürün mimarisine bağlıdır.

Process Environment Block (PEB) Nedir ve Neden Kritik

Process Environment Block (PEB), Windows üzerinde çalışan her sürece ait olan ve tamamen kullanıcı-modu adres alanında bulunan bir veri yapısıdır. Bir sürecin nasıl başlatıldığına, hangi parametrelerle çalıştığına ve çalışma ortamına dair birçok kritik metadata bu yapı içinde tutulur. Güvenlik araçlarının büyük bir bölümü, süreçlere dair bilgileri doğrudan veya dolaylı olarak PEB üzerinden okur.

PEB içerisinde yer alan başlıca alanlar şunlardır:

Alan	Açıklama
CommandLine	Sürecin hangi komut satırı ile başlatıldığı
ImagePathName	Çalıştırılan binary'nin tam yolu
Environment	Süreç ortam değişkenleri
Ldr	Yüklenmiş modülleri tutan loader yapıları
ProcessParameters	Süreç başlangıç parametrelerini içeren yapı

> ⚠ **Kritik Nokta: PEB, çekirdek tarafından salt okunur bir yapı değildir. Kullanıcı-modu adres alanında bulunur ve yeterli yetkiye sahip bir süreç tarafından okunabilir ve değiştirilebilir.**

Bu ayrım pratikte şuna yol açar:

- Kernel, sürecin başlangıç koşullarını sabit bir snapshot olarak görür.
- User-mode araçlar, saldırgan tarafından değiştirilebilen runtime metadata'ya bakar.
- Sonuç olarak, aynı süreç farklı telemetri kaynaklarında farklı "gerçeklikler" olarak algılanabilir.

> ☐ **Teknik Açıklama:** PEB manipülasyonu, sürecin gerçekten çalıştırdığı kodu değiştirmez. Değiştirilen şey, güvenlik araçlarının ve analizcilerin referans aldığı **süreç metadata'sıdır**. Bu nedenle PEB manipülasyonu bir çalıştırma (execution) tekniği değil; gözlemlenebilirliği bozma (deception) tekniğidir.

Kernel-User Telemetri Ayrımı ve Zamanlama Penceresi

PEB manipülasyonunu mümkün kılan şey karmaşık bir exploit zinciri değil, Windows'un süreç oluşturma modelinde yer alan doğal bir zamanlama farkıdır. Süreç oluşturma sırasında çekirdek ve kullanıcı-modu bileşenleri farklı aşamalarda devreye girer ve bu durum, aynı sürece ait telemetrinin farklı zamanlarda üretilmesine neden olur.

Bu akış kabaca şu şekilde ilerler:

Adım	Ne Olur	Durum
T-0	Çekirdek süreci oluşturur; image path, command line ve parent PID belirlenir	✓ Sabit
T-1	Kernel callback'leri ve ETW event'leri tetiklenir; oluşturma anına en yakın telemetri üretilir	✓ Sabit
T-2	Kullanıcı-modu initialization başlar PEB doldurulur, thread'ler başlar	⚠ Değişken
T-3	User-mode araçlar PEB üzerinden veri okur; süreç artık gözlemlenebilir hâle gelir	⚠ Manipüle edilebilir

☐ Kritik Zamanlama Penceresi

Çekirdek telemetrisi üretilir → Sysmon Event ID 1 loglanır → Ancak user-mode zenginleştirme henüz yapılmamış veya farklı bir anda yapılmış olabilir. Bu kısa ama yeterli pencerede saldırgan PEB içindeki belirli alanları manipüle edebilir.

Sonuç: Çekirdek kaynaklı loglar sürecin oluşturulduğu andaki durumu temsil ederken, user-mode araçlar süreç çalıştıktan sonra gözlemlenen durumu yansıtır. Bu bir yarış durumu değil; iki farklı gözlem noktasının zaman içinde birbirinden kopmasıdır. **Loglar yanlış değildir**, yalnızca aynı olayı farklı katmanlardan ve farklı zaman durumlarından raporlar.

Telemetry Matrix: Sürecin Hangi Anını Temsil Eder?

Bu çalışmanın merkezindeki soru şudur: “Bir süreç hakkında gördüğüm bilgi, o sürecin hangi zaman dilimine aittir?”

Loglara çoğu zaman doğru / yanlış perspektifiyle yaklaşılır. Oysa Windows telemetrisi söz konusu olduğunda asıl belirleyici olan **zamanlama ve gözlemin yapıldığı katmandır**.

Kaynak	Katman	Ne Zaman Okunur	Ne Gösterir	Manipülasyon Riski	Güven
Kernel ETW (ProcessStart)	Kernel	Oluşturma anı	Gercek Parent ve Command line	Çok zor	★★★★★
PsSetCreateProcessNotifyRoutineEx	Kernel	Oluşturma anı	Ham süreç oluşturma bilgisi	Çok zor	★★★★★
Sysmon Event ID 1	Kerne→User	Oluşturma anı	CLI, parent snapshot	Zamanlama bağımlı	★★★
EDR Process Tree (UI)	Hibrit	Oluştuktan sonra	Zenginleştirilmiş süreç görünümü	Orta	★★★
Process Explorer/ Task Manager	User-mode	Çalışma anı	Anlık PEB durumu	Yüksek	★★
Memory Forensics	Offline	Dump alındığı an	O anki bellek durumu	Senaryoya bağlı	★★★
Sysmon Event ID 10	Kernel	Runtime	Süreçler arası etkileşim	Düşük	★★★★

Not: Bu matris, teknik dokümanlar, açık kaynak araştırmalar, blog yazıları ve laboratuvar testlerinden elde edilen bulgular birleştirilerek hazırlanmıştır. Sysmon Event ID 1 için güven seviyesi, kernel callback tetiklemesi ile user-mode zenginleştirme arasındaki zamanlama ve konfigürasyona bağlı olarak değerlendirilmiştir.

2. Process Environment Block (PEB) Yapısı

```
┌──────────────────────────────────────────────────────────────────────┐
│                        THREAD (fs/gs register)                       │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │  TEB (Thread Environment Block)                              │   │
│  │  Offset: fs:[0x30] (x86) / gs:[0x60] (x64)                   │   │
│  │                                                              │   │
│  │   ┌──────────────────────┐                                   │   │
│  │   │ +0x000  ExceptionList │                                   │   │
│  │   │ +0x008  StackBase     │                                   │   │
│  │   │ +0x010  StackLimit    │                                   │   │
│  │   │ +0x030  PEB ──────────┼──┐  ←  Pointer to PEB            │   │
│  │   └──────────────────────┘  │                                 │   │
│  └──────────────────────────────┼─────────────────────────────────┘   │
└──────────────────────────────────┼──────────────────────────────────────┘
                                   ▼
┌──────────────────────────────────────────────────────────────────────┐
│                PEB (Process Environment Block)                       │
│                Base Address: 0x00007FF...                            │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ +0x000  BYTE Reserved1[2]                                    │   │
│  │ +0x002  BYTE BeingDebugged        ⚠ Anti-debug flag          │   │
│  │ +0x003  BYTE Reserved2[1]                                    │   │
│  │ +0x008  PVOID Reserved3[2]                                   │   │
│  │ +0x018  PPEB_LDR_DATA Ldr ──────────────┐                    │   │
│  │ +0x020  PRTL_USER_PROCESS_PARAMETERS    │                    │   │
│  │         ProcessParameters ──────────┐   │                    │   │
│  │ +0x010  ImageBaseAddress            │   │                    │   │
│  └──────────────────────────────────────┼───┼────────────────────┘   │
└──────────────────────────────────────────┼───┼──────────────────────────┘
                                           ▼   ▼
┌──────────────────────────────────┐   ┌──────────────────────────────┐
│  RTL_USER_PROCESS_PARAMETERS     │   │  PEB_LDR_DATA                │
│  ⋆ SALDIRGANIN HEDEFİ!           │   │  (Loader Data)               │
│                                  │   │                              │
│ +0x000 MaximumLength             │   │ +0x00 Length                 │
│ +0x004 Length                    │   │ +0x04 Initialized            │
│ +0x008 Flags                     │   │ +0x08 SsHandle               │
│ +0x010 Environment               │   │ +0x10 InLoadOrderModuleList  │
│ +0x020 WindowTitle               │   │       ├─ ntdll.dll           │
│ +0x030 ImagePathName             │   │       ├─ kernel32.dll        │
│ +0x040 CurrentDirectory          │   │       └─ user32.dll          │
│ +0x070 CommandLine ⚠             │   │ +0x18 InMemoryOrderModList   │
│        └─> UNICODE_STRING        │   │ +0x20 InInitOrderModList     │
│            Buffer ──────┐        │   │                              │
│                         │        │   │  ⚠ DLL Unlinking hedefi      │
│                         ▼        │   └──────────────────────────────┘
│  ┌──────────────────────┐        │
│  │ Actual Command Line: │        │
│  │"certutil.exe -hashfile"│      │  ← Burası manipüle ediliyor!
│  └──────────────────────┘        │
└──────────────────────────────────┘
```

🎯 Saldırganın Hedefi:
ProcessParameters->CommandLine.Buffer

Laboratuvar Bulguları: Sistem Üzerinde Doğrulama

Bu çalışmada anlatılan telemetri desenkronizasyonunu doğrulamak amacıyla, iki farklı senaryo üzerinden kontrollü laboratuvar testleri gerçekleştirildi. Senaryolar, aynı tekniğin hem analiz ortamlarında görünür hem de gerçek dünya saldırı senaryolarında daha sessiz nasıl davrandığını gösterecek şekilde tasarlandı.

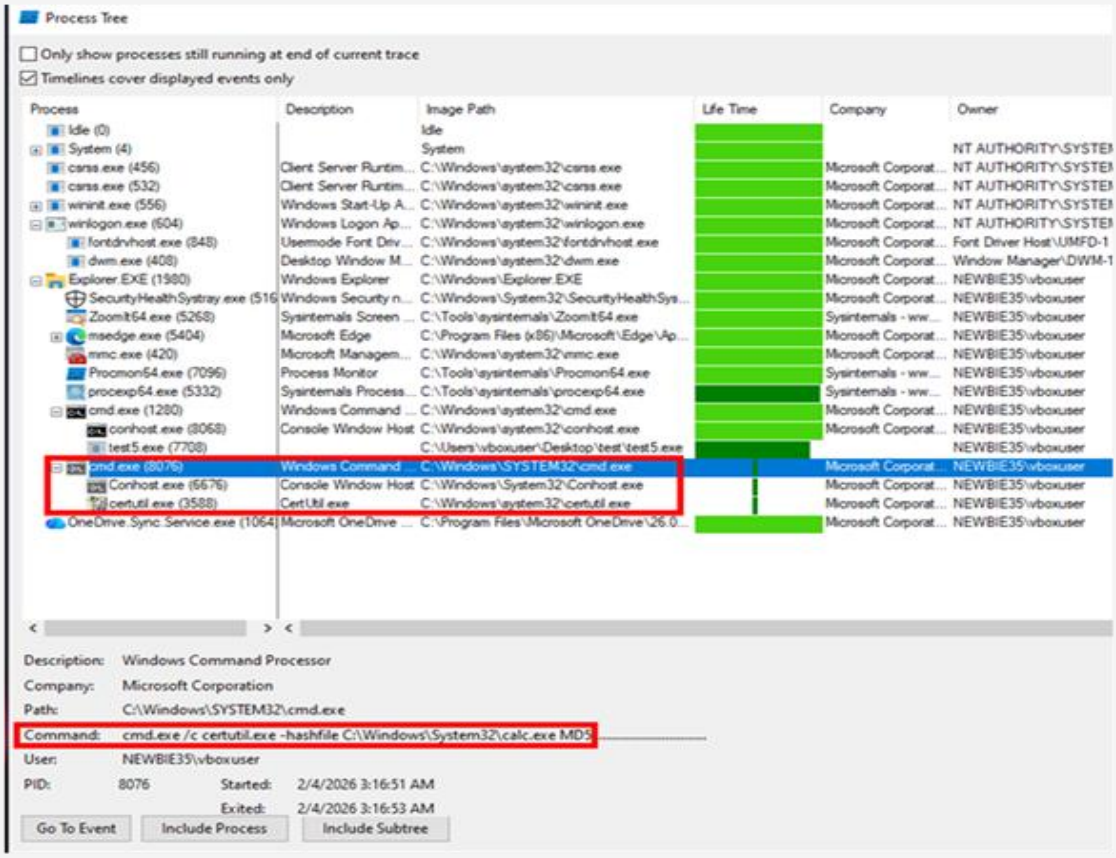
Senaryo 1: Görünür Çocuk Süreç (Gösteri Modu)

Tekniğin etkisini gözle görülür hâle getirmek için, bilinçli olarak çocuk süreç oluşturan bir senaryo kurgulandı. Süreç ağacı ve Sysmon Event ID 1 kayıtları ilk bakışta tutarlı ve zararsız bir akış sergilemektedir. Ancak detaylı incelemede, ebeveyn sürecin komut satırı ile çocuk sürecin gerçekleştirdiği aktiviteler arasında davranışsal bir tutarsızlık oluştuğu görülmektedir.

Bu senaryoda, EDR ve sandbox analizleri sırasında kötü niyetli komut yürütmesi süreç ağacı üzerinde hâlâ gözlemlenebilmektedir. Amaç, tekniğin temel etkisini açık ve takip edilebilir bir biçimde ortaya koymaktır.

Process Tree Analizi (Procmon)

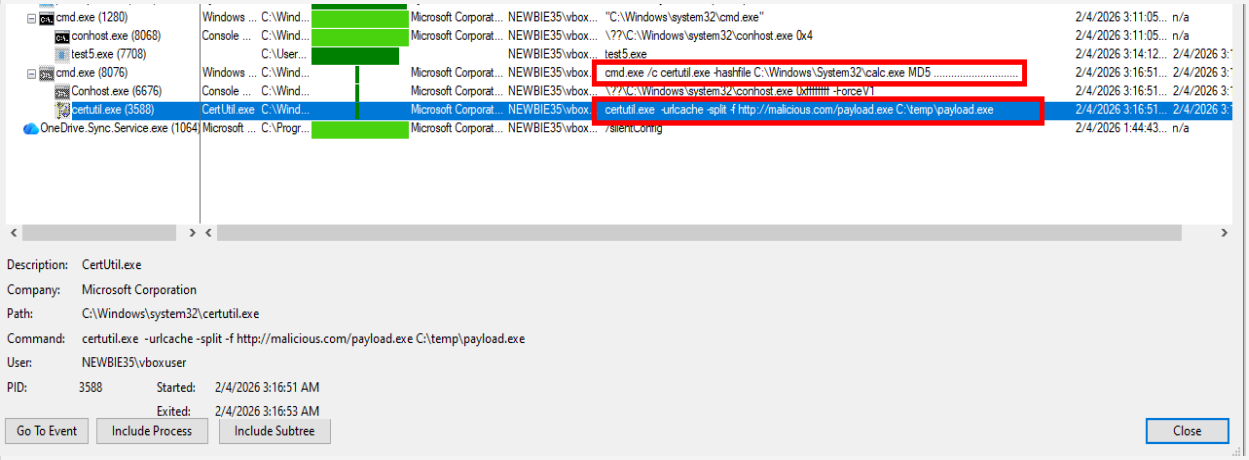
Lab Gözlemi: aşamada süreç ağacı üzerinde belirgin bir şüpheli durum gözlemlenmemektedir. cmd.exe üzerinden çalıştırılan komut açıkça görülmekte; certutil.exe sürecinin calc.exe dosyasının hash değerini aldığı anlaşılmaktadır. Bu bağlamda, certutil.exe'nin çocuk süreç olarak görünmesi süreç ağacı açısından tutarlı ve olağan bir davranış sergilemektedir.



Görsel 1 - Procmon

Process Tree Analizi (Procmon)

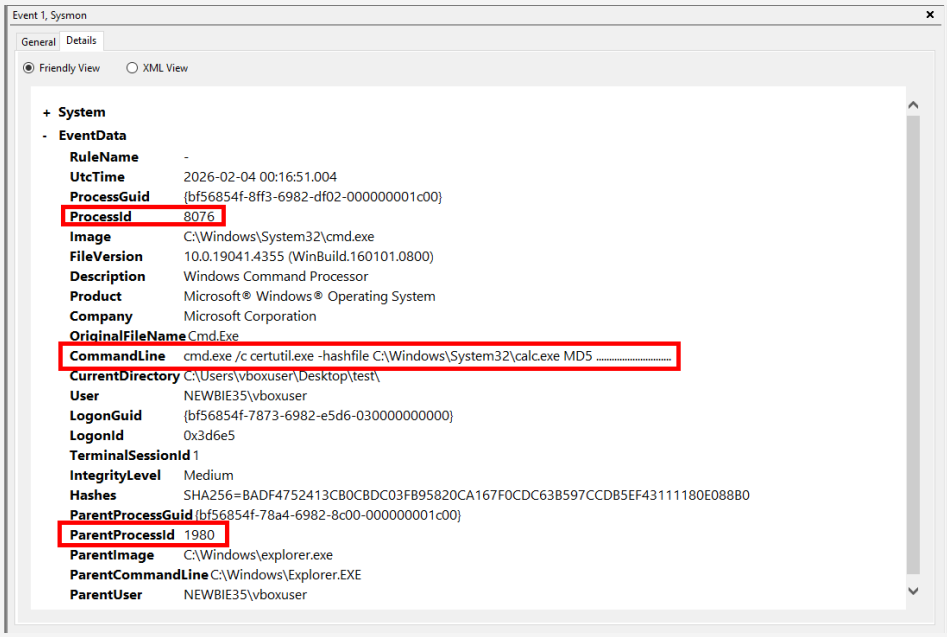
Lab Gözlemi: Süreç ağacı yüzeyde tutarlı görünmesine rağmen, parent process'in komut satırı ile child process'in yürüttüğü işlemler arasında anlamsal bir kopukluk bulunmaktadır. Bu görünürlük, senaryonun bilinçli olarak çocuk süreç oluşturacak şekilde kurgulanmasından kaynaklanmaktadır.



Görsel 2 - Procmon

Sysmon Event ID 1 - İlk Süreç Oluşturma

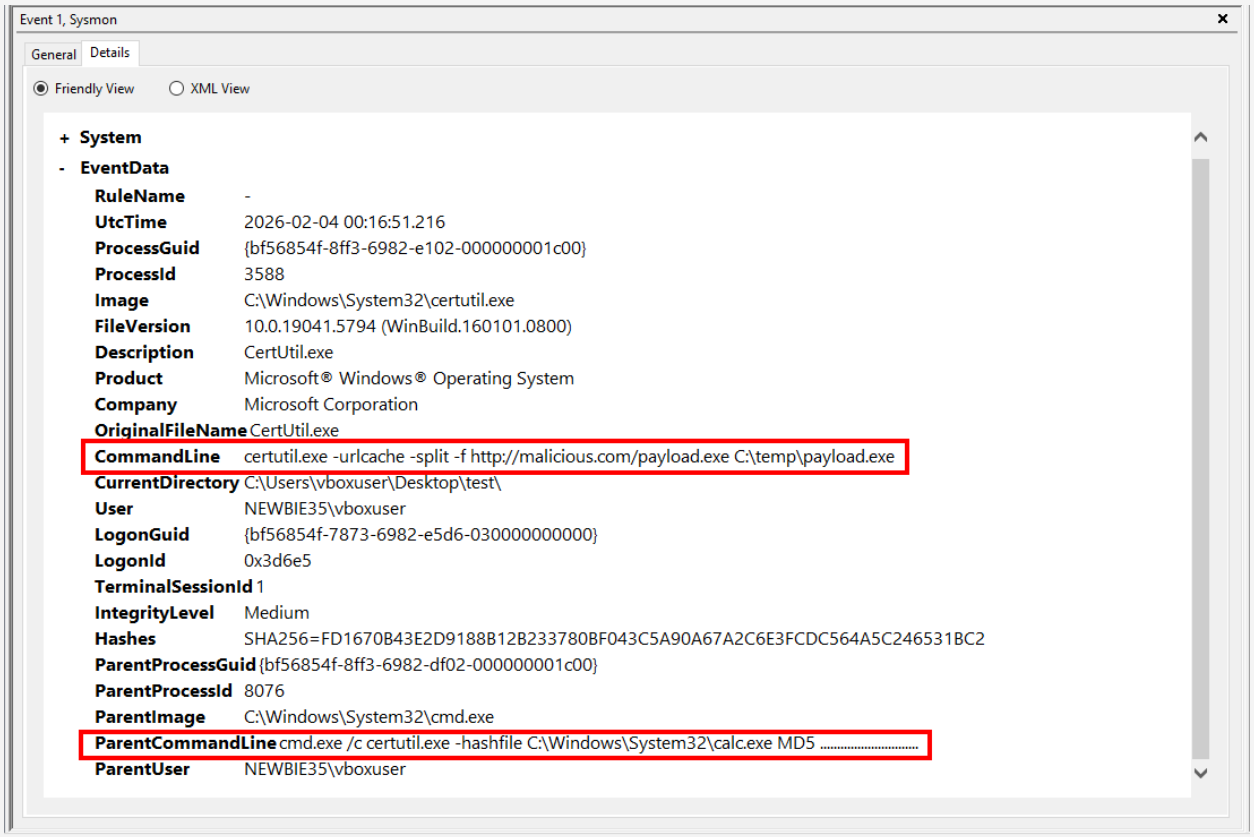
Lab Gözlemi: Sysmon kayıtları olağan ve zararsız bir davranış sergilemektedir. Loglarda yalnızca certutil.exe kullanılarak bir dosyanın hash değerinin hesaplandığı görülmektedir.



Görsel 3 - Sysmon log

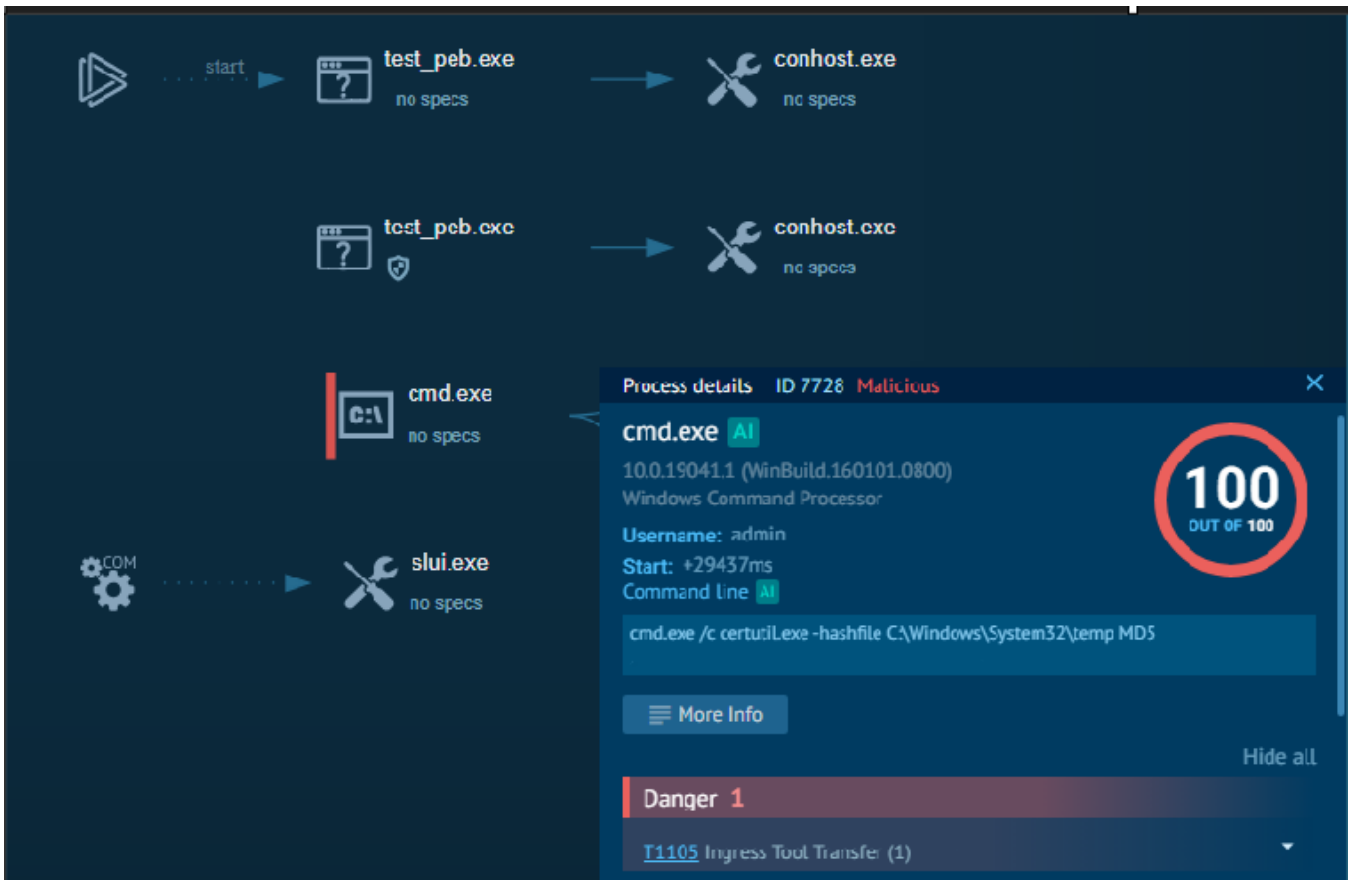
Sysmon Event ID 1 - İlk Süreç Oluşturma

Lab Gözlemi: Aktivitenin bilinçli olarak ayrı bir çocuk süreç şeklinde çalıştırılması, ek bir Sysmon olayı üretilmesine yol açmıştır. Bu olay arka plandaki yürütmeyi görünür kılsa da, parent process'in command line bilgisi ile gerçek yürütme arasında anlamsal bir kopukluk bulunmaktadır.

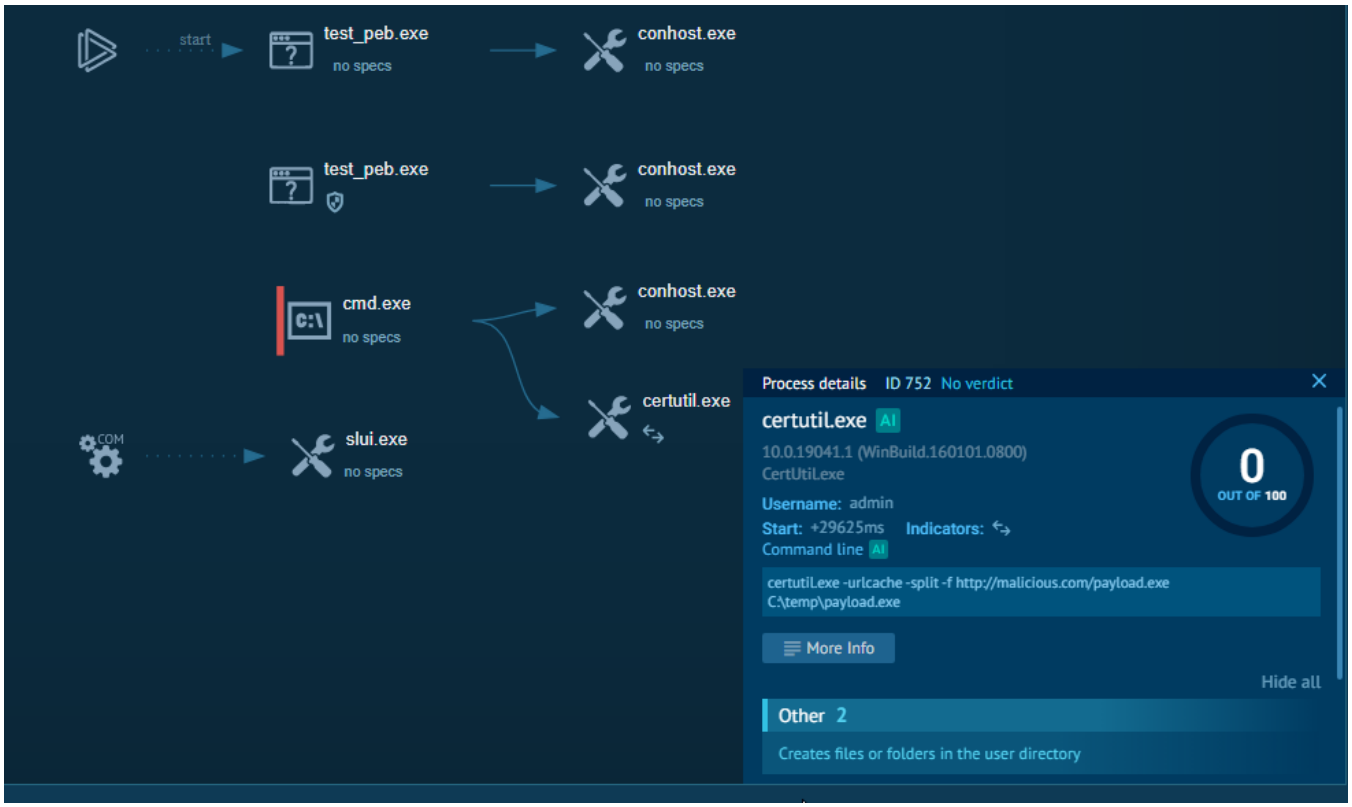


Görsel 4 - Sysmon log

ANYRUN



Görsel 5 - AnyRun



Görsel 6 - AnyRun

Senaryo 2: Gizli Yürütme (Gerçek Saldırı)

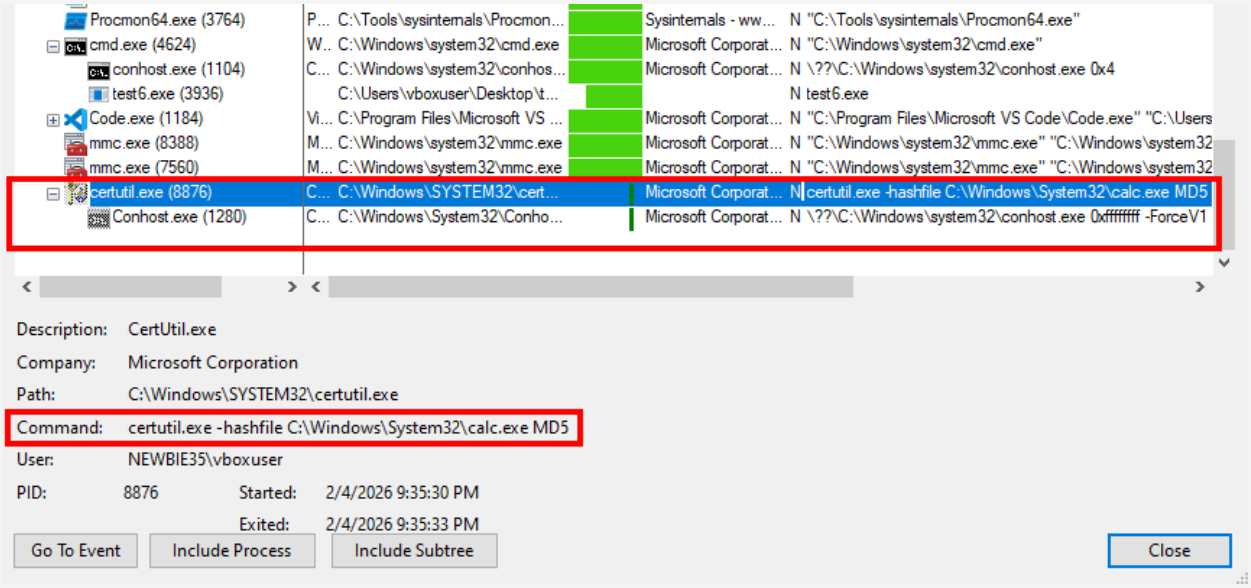
Bu senaryoda herhangi bir ek çocuk süreç oluşturulmamış, yürütme mevcut süreç içerisine gerçekleştirilmiştir. Procmon süreç ağacında kötü niyetli aktiviteye dair bir iz görülmezken, Sysmon yalnızca tek bir Event ID 1 kaydı üretmiştir. Varsayılan Sysmon yapılandırması altında, süreç oluşturma telemetrisi bu yürütmeyi ortaya koymakta yetersiz kalmaktadır.

Bu senaryo, süreç oluşturma telemetrisine dayalı tespitlerin neden tek başına yeterli olmadığını açıkça göstermektedir.

(Not: Kernel ETW açıksa veya Sysmon Event ID 10/11 gibi ek telemetri toplanıyorsa farklı sonuçlar elde edilebilir.)

Süreç Ağacı Analizi (Procmon)

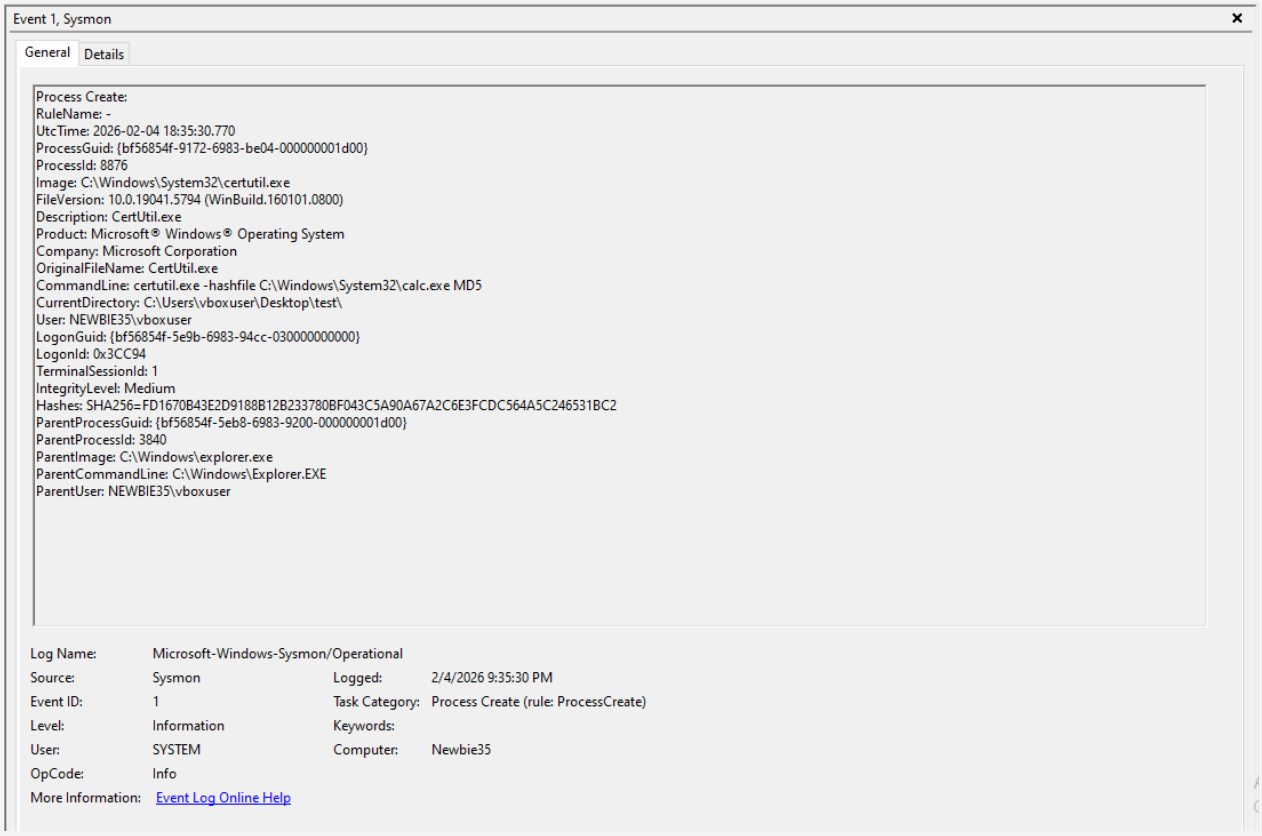
Lab Gözlemi: Procmon üzerinden yapılan süreç ağacı analizinde, arka planda gerçekleşen yürütmeye ait herhangi bir çocuk süreç veya komut görünürlüğü tespit edilmemiştir.



Görsel 7- Procmon

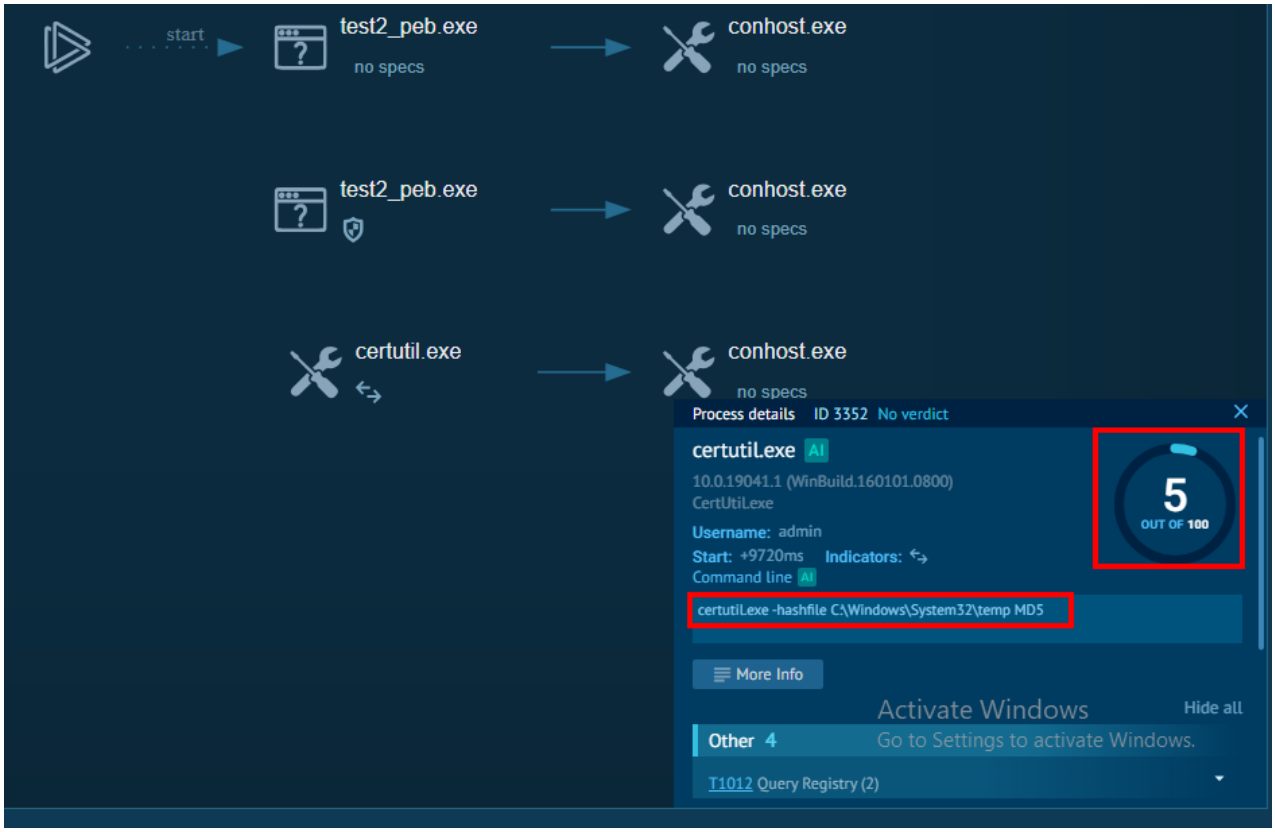
Sysmon Event ID 1 - İlk Süreç Oluşturma

Lab Gözlemi: Sysmon telemetrisi, bu senaryoda yalnızca tek bir Event ID 1 kaydı üretmiş; gerçekleştirilen yürütmeye dair ek bir log sağlamamıştır.



Görsel 8 – Sysmon

AnyRun



Görsel 9 – AnyRun

Görüldüğü üzere zararlı komut herhangi bir yerde görünmedi ve önceki testlerimizin aksine bu sefer zararlı aktivite tespit edilmedi. Süreç boyunca zararlı aktivite hedeflendiği gibi çalıştı.

Görsel 10 – AnyRun

EDR Bu Tekniği Neden Bazen Yakalıyor, Bazen Sessiz Kalıyor?

PEB manipülasyonu gibi tekniklerde EDR davranışının tutarsız görünmesinin temel nedeni, süreç telemetrisinin tek bir katmandan değil, iki farklı düzlemden toplanmasıdır: kernel ve user-mode.

Kernel tarafında, Windows süreç oluşturma sırasında kayıtlı callback'lere `PS_CREATE_NOTIFY_INFO` yapısını iletir. Bu yapıdaki **`CommandLine`** alanı, sürecin oluşturulduğu andaki gerçek komut satırını temsil eder. EDR sürücüsü bu callback'i kullanıyorsa, bu bilgi kullanıcı-modu bellek yapılarından bağımsız olarak alınır ve PEB sonradan değiştirilse bile etkilenmez.

User-mode tarafında ise süreç ağacı, komut satırı ve görselleştirme verileri çoğunlukla PEB veya PEB'den okuma yapan API'ler üzerinden zenginleştirilir. Bu okuma, süreç oluşturulduktan sonra gerçekleştiği için, saldırganın PEB üzerinde yaptığı değişiklikler bu katmana doğrudan yansır.

Bu ayrım, EDR'nin neden bazı ortamlarda bu tekniği tespit edebildiğini, bazılarında ise tamamen sessiz kaldığını açıklar

EDR, kernel'dan aldığı oluşturma-anı komut satırı ile user-mode'da sonradan okuduğu komut satırını karşılaştırıyorsa, aynı süreç için ortaya çıkan tutarsızlık güçlü bir tespit sinyali üretir. Ancak pratikte birçok EDR bu korelasyonu otomatik olarak gerçekleştirmez ya da kernel verisini yalnızca arka planda tutup arayüzde göstermemeyi tercih eder.

Davranışsal açıdan bakıldığında PEB manipülasyonu, doğası gereği bir sürecin başka bir sürecin belleğine yazmasını gerektirir. **NtWriteVirtualMemory** ve benzeri çağrılarla yapılan bu erişimler, Sysmon Event ID 10 gibi "process access" telemetrilerinde gözlemlenebilir. EDR bu olayları süreç ilişkileri, zamanlama ve LOLBin kullanımıyla birlikte değerlendirdiğinde, PEB yazma davranışı üzerinden tespit üretebilir.

Ancak EDR yalnızca user-mode kaynaklı komut satırına güveniyor, kernel tarafındaki oluşturma-anı verisini kullanmıyor ya da bu iki katmanı ilişkilendirmiyorsa, PEB manipülasyonu hiçbir alarm üretmeden görünmez kalabilir..

Kısaca, tekniğin yakalanması yalnızca ürün kapasitesine bağlı değildir; verilerin hangi katmandan, hangi anda alındığı ve bunların birbirleriyle nasıl ilişkilendirildiği belirleyicidir.

5. EDR Bypass Teknikleri ve Savunma Katmanları

```
═══════════════════════════════════════════════════════════════
                    EDR BYPASS MATRİSİ
═══════════════════════════════════════════════════════════════

┌─────────────────────────────────────────────────────────────┐
│ TEKNİK 1: PEB MANIPULATION                                  │
├─────────────────────────────────────────────────────────────┤
│ Hedef: ProcessParameters->CommandLine                       │
│                                                             │
│ Bypass Edilen:                                              │
│   ✗ Sysmon Event ID 1 (tarihsel log)                        │
│   ✗ User-mode process monitoring                            │
│   ✗ SIEM correlation rules (benign command görüyor)         │
│                                                             │
│ Bypass Edilemeyen:                                          │
│   ✓ Kernel callbacks (real-time validation)                 │
│   ✓ Memory forensics (VAD tree analysis)                    │
│   ✓ Cross-validation (Sysmon vs live process)               │
│                                                             │
│ Gerçek Kullanım: APT41, Cobalt Strike, LockerGoga           │
└─────────────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────────────┐
│ TEKNİK 2: DLL UNLINKING                                     │
├─────────────────────────────────────────────────────────────┤
│ Hedef: PEB.Ldr->InMemoryOrderModuleList                     │
│                                                             │
│ Yöntem:                                                     │
│   1. Reflective DLL injection                               │
│   2. DLL'i PEB.Ldr listelerinden çıkar:                     │
│      - InLoadOrderModuleList                                │
│      - InMemoryOrderModuleList                              │
│      - InInitializationOrderModuleList                      │
│   3. Process Explorer artık DLL'i görmez                    │
│                                                             │
│ Bypass Edilen:                                              │
│   ✗ Process Explorer (PEB.Ldr okur)                         │
│   ✗ Task Manager (module enumeration)                       │
│   ✗ User-mode monitoring tools                              │
│                                                             │
│ Bypass Edilemeyen:                                          │
│   ✓ VAD tree analysis (kernel memory map)                   │
│   ✓ Memory dumps (DLL hala bellekte)                        │
│   ✓ Sysmon Event ID 7 (ImageLoad - oluşturulma anı)         │
│                                                             │
│ Gerçek Kullanım: Cobalt Strike Beacon, APT29 WellMess       │
└─────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────┐
│ TEKNİK 3: CALL STACK SPOOFING                                    │
├──────────────────────────────────────────────────────────────────┤
│ Hedef: Stack walk deception                                      │
│                                                                  │
│ Yöntem:                                                          │
│   1. Sahte stack frame'ler oluştur                               │
│   2. Return address'i ntdll.dll'e point ettir                    │
│   3. EDR stack walk meşru origin gösterir                        │
│                                                                  │
│ EDR Stack Walking:                                               │
│   ┌──────────────────────────────────────┐                       │
│   │ VirtualAllocEx() called              │                       │
│   │   ↓ Stack trace:                     │                       │
│   │   ├─ ntdll.dll!NtAllocateVirtualMem  │ ✅ Meşru              │
│   │   ├─ kernel32.dll!VirtualAllocEx     │ ✅ Meşru              │
│   │   └─ [spoofed frame]                 │ ⚫ Sahte              │
│   │                                      │                       │
│   │ EDR Decision: ALLOW (meşru görünüyor)│                       │
│   └──────────────────────────────────────┘                       │
│                                                                  │
│ Bypass Edilen:                                                   │
│   ❌ EDR stack walking                                           │
│   ❌ Behavioral analysis (meşru origin)                          │
│                                                                  │
│ Bypass Edilemeyen:                                               │
│   ✅ Kernel-level stack validation                               │
│   ✅ Return address consistency checks                           │
│   ✅ User vs kernel mode call stack comparison                   │
│                                                                  │
│ Gerçek Kullanım: APT41 DodgeBox, Cobalt Strike 4.5+              │
└──────────────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────────┐
│ TEKNİK 4: EDR SILENCER (WFP Filters)                             │
├──────────────────────────────────────────────────────────────────┤
│ Hedef: EDR telemetrisini kapatmak                                │
│                                                                  │
│ Windows Filtering Platform (WFP) Abuse:                          │
│   1. WFP filter kaydedilir                                       │
│   2. EDR network callback'leri bloklanır                         │
│   3. Process/thread creation bildirimleri drop edilir            │
│   4. File system eventleri filtrelenir                           │
│                                                                  │
│ Etki:                                                            │
│   ┌──────────────────────────────────────┐                       │
│   │ ÖNCESI:                              │                       │
│   │ ✅ Process created → EDR alerted     │                       │
│   │ ✅ Network connection → EDR logged   │                       │
│   │ ✅ File modified → EDR detected      │                       │
│   └──────────────────────────────────────┘                       │
│                                                                  │
│   ┌──────────────────────────────────────┐                       │
│   │ SONRASI (WFP filter aktif):          │                       │
│   │ ❌ Process created → EDR BLIND       │                       │
│   │ ❌ Network connection → EDR BLIND    │                       │
│   │ ❌ File modified → EDR BLIND         │                       │
│   └──────────────────────────────────────┘                       │
│                                                                  │
│ Bypass Edilemeyen:                                               │
│   ✅ Kernel driver self-protection                               │
│   ✅ Telemetry redundancy (çoklu toplama)                        │
│   ✅ WFP filter audit (Event ID 5157)                            │
│                                                                  │
│ Gerçek Kullanım: EDRSilencer (2024), Targeted ransomware         │
└──────────────────────────────────────────────────────────────────┘
```

Tespit: Loglar Yanlış Değil Ama Eksik

PEB manipülasyonu gibi tekniklerde temel problem log eksikliği değildir. Loglar üretilir, SIEM'e düşer ve çoğu zaman "zararsız" görünür. Sorun, bu logların aynı gerçeğin farklı anlarını temsil etmesidir. Bir süreç için oluşturma anında kaydedilen bilgiler, çalışma sırasında okunan metadata ve gerçek davranışı yansıtan olaylar aynı zaman çizgisinde yer almaz. Tespit açığı tam olarak burada oluşur.

Tek Kaynaklı Tespit Neden Başarısız Oluyor?

Süreç analizlerinin çoğu şu varsayıma dayanır: "Süreç oluşturulurken kaydedilen bilgi, süreç çalışırken de geçerlidir." Bu doğru değildir. Windows'ta bazı bilgiler kernel tarafından oluşturma anında kaydedilir, bazıları ise süreç çalışırken user-mode bellekten okunur. PEB manipülasyonu bu ikinci grubu hedef alır. Bu nedenle Sysmon, EDR arayüzü ve diğer user-mode araçlar çelişkili sonuçlar gösterebilir; bu bir yazılım hatası değil, mimari bir farktır.

Asıl Sinyal: Tutarsızlık

Bu teknikleri yakalamanın yolu yalnızca "şüpheli bir event" aramak değildir. Asıl sinyal, kaynaklar arasındaki tutarsızlıktır. Benign komut satırıyla başlayan bir süreç, uzun süre yaşayan LOLBin davranışı, süreç içinden beklenmeyen erişimler ve oluşturma anı ile çalışma anı arasındaki anlam farkı bunlar tek başına zararlı değildir; birlikte değerlendirildiğinde anlamlıdır.

> Soru şudur: "Bu süreç gerçekten bildirdiği gibi mi davranıyor?"

ETW Neden Ayrı Bir Yerde Konumlanır?

ETW tabanlı kernel telemetrisi, süreç oluşturma anını user-mode manipülasyonlardan önce gözlemler; parent ilişkisini ve zamanlamayı daha ham hâliyle yakalar. Bu nedenle PEB manipülasyonu ve parent spoofing gibi teknikler Sysmon'da tutarlı, EDR arayüzünde makul görünebilirken, ETW tarafında kapsam dışı olarak ortaya çıkabilir. ETW tek başına yeterli değildir; ancak yanıltılması daha zor bir referans noktasıdır.

Davranış > Metadata

Komut satırı, parent process veya image path tek başına güvenilir değildir. Buna karşın şu sorular çok daha değerlidir: Bu süreç neden başka bir sürecin belleğine erişti? Neden network trafiği üretti? Neden normalde yapmadığı bir API çağrısını gerçekleştirdi? Sysmon Event ID 10 gibi süreç içi etkileşimleri gösteren loglar, oluşturma anı telemetrisinden daha geniş kapsam sunar.

> Bölümün net çıktısı:
>
> - Loglar yalan söylemez, ama eksik bağlamla konuşur
> - PEB manipülasyonu logları silmez, algıyı bozar
> - Etkili tespit, tek bir event değil ilişki ve korelasyon analizi gerektirir
> - Kernel ve user-mode veriler ayrı ayrı değil birlikte anlamlıdır

Savunma Mimarisi: Tek Loga Değil, Katmanlara Odaklan

Bu tür saldırılar tek bir log ile tespit edilemez; saldırgan bunu bilerek tek loga güvenilmesini sağlar. Etkili savunma, yalnızca event'lere değil, katmanlı analiz ve korelasyona dayanır.

1- Oluşturma Anı (Ground Truth)

- Kernel ETW
- Process notify callback'leri

→ Bu süreç gerçekten nasıl başladı?

2- Snapshot Loglar

- Sysmon Event ID 1
- EDR create events

→ O an bize ne söylendi?

3- Runtime Davranış

- Sysmon Event ID 10
- API çağrıları
- Process injection sinyalleri

→ Bu süreç çalışırken ne yaptı?

4- Durum Analizi

- Parent-child ilişkileri
- LOLBin zincirleri
- Zaman korelasyonu

→ Bu davranış mantıklı mı?

5- Canlı Doğrulama

- EDR live response
- Process Explorer
- Memory snapshot

→ Şu an ne görüyorum?

6□ Çapraz Kaynak Kontrolü

- Kernel ↔ User
- Log ↔ Bellek
- Event ↔ Davranış

→ *Bu kaynaklar aynı şeyi mi söylüyor?*

⚠ Kritik: Bu katmanlardan biri ya da birkaçı eksik olsa bile sistem çalışmaya devam eder; saldırgan ise tam olarak bu boşlukları kullanır.

BlackCat (ALPHV) Ransomware-as-a-Service

Hedef: Healthcare Provider (hastane zinciri) - Ekim 2022

Etki:	8 hastane offline, HIPAA breach, $22M hasar
Tespit Süresi:	Immediate (users reported, systems down)
EDR Alarm:	0 (EDR killed at Stage 3)
Downtime:	11 gün (critical systems), 2.1M patient records

Saldırı Zinciri

Aşama	Teknik	Bypass Edilen	Detay
Ik Erişim	**Compromised** **RDP Brute Force**	• Failed login alerts bypass • Geo-anomaly detection bypass	Citrix Gateway brüte force Zayif parola, No MFA
Reconnaissan + PEB Manipulatıon	**Domain Enumeration** **CommandLine Spoofing**	• Sysmon benign cmdline • SIEM recon rules bypass	nltest, net commands Gerçek: nltest /dclist
LSASS Dump	**PEB + Call Stack** **Combo Attack**	• Event ID 10 logged • EDR LSASS alert bypass	rundll32 comsvcs.dll Benign cmdline + meşru call stack
EDR Killer	**BYOVD** **+ WFP Filter**	• TÜM EDR TELEMETRİSİ	RTCore64.sys vulnerable driver Kernel-level EDR terminate
Lateral Movement	**PSExec + SMB** **PEB maipulation** **her bir geçişte**	• Lateral movement (EDR kör)	Stolen credentials
Backup Destruction	**Shadow Copy Delete** **Veeam Targeting**	• Backup monitoring (EDR kör)	vssadmin delete shadows /a l l Veeam backup server access
Ransomware	**BlackCat Rust Binary** **Multi-thread** **Encryption**	• File integrity • Mass encryption (EDR kör)	2.1M patient records DICOM files encrypted $10M Monero demand

APT41 - LockerGoga Ransomware

Hedef: Norsk Hydro - Mart 2019

Etki:	40+ fabrika kapatıldı, $71 milyon hasar
Tespit Süresi:	72+ saat (manuel inceleme)
EDR Alarm:	0 (EDR Silencer aktif)
Toplam Süre:	ilk erişimden encryption'a 8 gün

Saldırı Zinciri

Aşama	Teknik	Bypass Edilen	Detay
Ik Erişim	**Spear-phishing + Weaponized Excel**	• Email Gateway • Sandbox	certutil.exe ile payload download + mshta.exe → HTA execution
PEB Manipulation	**CommandLine Spoofing NtWriteVirtualMemory**	• Sysmon Event ID 1 • EDR UI • SIEM korelasyon	Gerçek: certutil -urlcache C2 Görünen: certutil –hashfile MD5
DLL Unlinking	**Reflective DLL Injection PEB.Ldr Manipulation**	• Process Explorer • Task Manager • Module scanning	Cobalt Strike Beacon + PEB LDR InMemoryOrderModuleList'ten çıkarıldı
Call Stack Spoofing	**Stack Frame Manipulation ROP Gadgets**	• EDR stack walking • Behavioral analysis • API call detection	VirtualAllocEx → ntdll.dll Meşru call chain görünümü
Credential Harvesting	**LSASS Memory Dumping comsvcs.dll MiniDump**	• Sysmon Event ID 10 • LSASS access alerts • Credential theft	rundll32 + PEB manipulation Microsoft imzalı binary comsvcs.dll
Lateral Movement	**WMI + PsExec Her host'ta tekrar**	• Lateral movement • Admin activity • Network analytics	40+ server, 3,000+ endpoint Domain admin credentials
EDR Silencer	**WFP Filter Callback blocking**	• Process monitoring • Network callbacks • File events	EDR tamamen kör Telemetri %0
Ransomware Deployment	**LockerGoga AES-256 Multi-threaded**	• File Integrity • Behavioral detection • Mass file alerts	Shadow copy deletion

APT29 - NOBELIUM Enterprise Breach

Hedef: Fortune 500 Technology Company - Mayıs 2021

Etki:	Kaynak kod çalındı, 3 aylık C2 erişimi
Tespit Süresi:	3 ay (Microsoft Threat Intelligence notification)
EDR Alarm:	0 (EDR Silencer kullanılmadı - gerek kalmadı)
Çal■nan Veri:	2.3 TB kaynak kod + internal dokümanlar

Saldırı Zinciri

A■ama	Teknik	Bypass Edilen	Detay
Ik Eri■im	**Password Spray + MFA Bypass**	• MFA (legacy auth) • Geo-location alerts • Login throttling	Azure AD hesabı VPN üzerinden kurumsal ağ erişim
PEB + PPID Spoofing	**Dual Spoofing Parent + CommandLine**	• Sysmon • EDR process tree • SIEM parent-child	Parent: cmd.exe → svchost.exe Cmdline: encoded → legitscript.ps1
Memory-Only Payload	**Fileless Execution Reflective DLL + Unlinking**	• File-based AV/EDR • Signature detection • YARA rules	Cobalt Strike Beacon Filess XOR + AES encrypted
DLL Unhookir + Call Stack	**EDR Hook Removal Syscall Direct**	• EDR API monitoring • Function hooking • Call validation	ntdll.dll unhook Direct kernel syscall + ROP chain
Persistence + C2	**WMI Event Subscription DNS Tunneling**	• Registry monitoring • Network C2 signatures • DPI/IDS	Named pipe: msagent Domain fronting CDN
Credential Theft	**LSASS Dump + Token Manipulation**	• LSASS access detection • Credential theft • Privilege escalation	comsvcs.dll MiniDump SYSTEM + token duplicate SeDebugPrivilege
Data Exfiltration	**Steganography + HTTPS Encryption**	• DLP solutions • Network traffic analysis • Exfiltration rules	2.3 TB over 3 months PNG metadata embedding Legitimate cloud storage

APT Savunma Katmanlar Bypass Matrisi

Savunma Katmanı	APT41	APT29	BlackCat	Bypass Tekniği
Email Gateway	✓	-	-	----------
Sandbox Analysis	✓	-	-	Delayed execution
Sysmon Event ID 1	✓	✓	✓	**PEB manipulation**
EDR Process Tree	✓	✓	✓	**PEB + PPID spoofing**
SIEM Korelasyon	✓	✓	✓	**PEB Benign görünüm**
Process Explorer	✓	✓	-	**PEB DLL unlinking**
Task Manager	✓	✓	-	**PEB Module enumeration bypass**
EDR Stack Walking	✓	✓	✓	**Call stack spoofing**
Behavioral Analysis	✓	✓	✓	**Meşru API origin**
LSASS Access Detection	✓	✓	✓	**Benign cmdline + stack**
Lateral Movement Detection	✓	✓	✓	Stolen admin creds
Network C2 Signatures	✓	✓	-	DNS tunneling / Domain fronting
File Integrity Monitoring	✓	-	✓	EDR silencer aktif
DLP Solutions	-	✓	-	Steganography
TÜM EDR TELEMETR■S■	✓	-	✓	WFP Filter / BYOVD

Sonuç

Bu çalışmanın vardığı nokta tek bir teknikten ibaret değildir. PEB manipülasyonu, Sysmon'un sınırlamaları ya da certutil istismarı tek başına "asıl problem" değildir. Asıl problem, loglara mutlak doğru muamelesi yapılmasıdır.

Windows üzerinde bazı telemetri kaynakları süreç oluşturulurken kernel seviyesinde sabitlenir; bazıları ise süreç çalışırken user-mode bellek yapılarından okunur. Bu fark, zamanlama penceresi oluştuğunda savunma tarafında ciddi bir kör nokta yaratır. Aynı süreç için farklı araçların farklı bir gerçeklik sunabilmesi bu nedenle mümkündür..

> Senaryoların Ortak Noktası: Loglar yanlış değildir, eksiktir. Tek bir kaynağa bakıldığında "benign" görünen bir davranış, durum bilgisi genişletildiğinde açıkça zararlı hâle gelir.

Buradan çıkan temel çıkarımlar:

- Oluşturma anı logları, sürecin tamamını anlatmaz
- User-mode görünüm doğru olabilir, ancak gerçeğin kendisi olmayabilir
- Kernel telemetrisi daha dirençlidir, fakat tek başına karar mekanizması olamaz
- Bellek analizi en güvenilir referanstır; ancak anlık değildir.

Bu nedenle soru artık "Log doğru mu?" değildir.

Asıl soru: "Bu log, hangi zamanda ve hangi koşullarda doğrudur?"

Modern saldırıların amacı gizlenmek değil, algıyı ve kapsamı bozmaktır.

Etkili savunma; daha fazla kuraldan değil, daha fazla korelasyon, zamanlama bilinci ve çok kaynaklı analizden geçer.

Temel Çıkarımlar

- ✓ Tek bir telemetri kaynağı hiçbir zaman yeterli değildir
- ✓ Sysmon gibi oluşturma anına bağlı loglar, sürecin tamamını temsil etmez
- ✓ Zamanlama farkları saldırganlar için bir gizlenme penceresi yaratır
- ✓ Süreçler tekil olaylar değil, zincirler hâlinde değerlendirilmelidir
- ✓ Davranışsal analiz, imza ve IOC tabanlı tespitlerin önüne geçmiştir
- ✓ Gerçek zamanlı gözlem, geçmiş log analizinin yerini değil tamamlayıcısını oluşturur

Bu çalışmada gösterilen teknikler varsayımsal değildir; güncel saldırı zincirlerinin parçasıdır. PEB manipülasyonu, geleneksel log güven varsayımlarını hedef alır ve tek kaynağa dayalı tespitleri bilinçli olarak yanıltır. Bu nedenle etkili savunma, daha fazla kural yazmaktan değil, zamanlama farklarını ve telemetri tutarsızlıklarını yakalayabilen tespit mimarileri kurmaktan geçer.

Kaynaklar ve Referanslar

- Velociraptor: ETW Part 2 - Process Parent Spoofing (2021)
- Trail of Bits: What child is this? Process Reparenting in Microsoft Windows (2022)
- Trail of Bits: ETW internals for security research and forensics (2023)
- Elastic Security Labs: Kernel ETW is the best ETW (2024)
- Huntress: I Wish I Was a Little Bit Taller - Dealing with Imperfection in Intrusions
- Microsoft: Process Environment Block (PEB) structure
- Microsoft: PsSetCreateProcessNotifyRoutineEx2 / PS_CREATE_NOTIFY_INFO
- SANS ISC: Malicious Process Environment Block Manipulation